

Christian Parker

CXP220034

CS 6314.0U1 Summer 2024

Introduction

How Do Developers React to RESTful API Evolution? was authored by Shaohua Wang, Iman Keivanloo, and Ying Zou and published in 2014. The study analyzed discussion on Stack Overflow following updates to publicly available RESTful APIs. Their data and conclusions are presented for API providers who may be considering future modifications. The authors collected data on the evolution of 11 different RESTful APIs and separated changes into 21 change types. They retrieved sets of posts from Stack Overflow related to each API change and examined developer engagement on these posts. With this data, they found that *adding new methods* to APIs results in the most questions and views per change. *Deleting methods* results in the most engagement (responses and likes). In this paper, I provide a summary for the article followed by a critique of its methodology. My critique raises concerns regarding the following:

1. Data Collection
 - a. The majority of the data came from Twitter.
2. Data Separation
 - a. The study considers merged and divided methods as deleted and the methods that replace them added.
3. Ambiguous Conclusions
 - a. The study fails to clearly analyze why change types receive various levels of engagement.

Summary

The article begins by introducing RESTful web service APIs, why they evolve, and how this evolution can be problematic for client code developers who integrate RESTful APIs. Many organizations like Twitter are opening RESTful APIs for client code developers to integrate into their own applications or services. Providers will evolve their APIs for various reasons. When an API evolves, any client applications or services using said API must migrate to function with the most recent updates. This can cause difficulties for client developers who often have no knowledge of how these changes are occurring. To better understand the effects of API evolution on client developers, this article considers 11 web APIs. API documentation is used to track changes since source code for these APIs is rarely publicly available. Changes are broken into 21 distinct types and discussion on Stack Overflow is separated and analyzed by type.

The APIs considered were extracted from a list of most popular APIs in ProgrammableWeb. From this list, the authors kept only APIs with multiple versions and documentation for multiple versions. Any APIs with the same domain as another API in the list were either merged or discarded. The same was done for APIs from the same company. After filtering, all publicly available versions of each API were considered.

Among the 11 web APIs considered, only three have more than 1,000 Stack Overflow posts, Twitter (46,646), Tumblr (1,372), and OpenStreetMap (1,072). Thus, a large majority of the posts are from Twitter.

The authors present three research questions considered for this article:

What are the change types of web API evolution?

They begin by separating the changes into three categories, *API-level*, *method-level*, and *parameter-level*. After categorization, they found that the average proportion of changed elements (methods and parameters) between versions is 82%. They also found that the four most common changes are *Add Method*, *Delete Method*, *Change Method Name* and *Add Parameter*, with 41.52% of these changes being *Add Method*.

Which types of changes trigger more questions from client code developers?

To answer this question, the authors considered how many questions were generated on Stack Overflow per change of each type. They found that *Add Method* is the highest with an average of 63 questions per change. They also found that *method-level* changes result in more questions than *parameter-level* changes.

Which types of changes bring discussion on posted questions from developers?

For this section, answers, comments, and views on Stack Overflow questions were quantified. They are broken into three distinct measurements:

- **Score.** The difference between up-votes and down-votes.
- **View Count.** The number of times the question has been viewed.
- **Answer Count.** The number of answers for the question.

Only Stack Overflow posts from the top six change types calculated in the previous question are considered. The authors found that *Delete Method* questions receive the highest scores and most answers relative to other change types whereas *Add Method* questions receive higher views relative to every other change type. This suggests that communication and answers for deleted methods is easier to come by than added methods. The authors presume that deleting methods tends to break client applications which puts more pressure on client developers to find an immediate solution, thus resulting in higher engagement on *Deleted Method* questions.

Critique

Data Collection. The authors either failed to collect a broad range of datasets or failed to be sufficiently clear that their data is heavily weighted towards Twitter. Based on table 2 of their dataset, they retrieved 46,646 posts from Stack Overflow regarding Twitter API updates. If we aggregate the posts regarding every other API, we only have 3,751. Therefore, Twitter accounts for 92.6% of the total posts considered. Given the overwhelming proportion of posts from Twitter, the authors should have made it clear that their results are biased and therefore only valid for Twitter. Unfortunately, they do not address this, despite having a Threats to Validity section where they state that retrieving posts only from Stack Overflow might bias their results.

Data Separation. The authors consider merged, divided, or deprecated methods as deleted from the previous version and the methods that replace them as added in the updated version. This concerns me because a merged method and a new method are fundamentally different changes. In the case of a merged method, there is no new functionality added, whereas a new method is entirely new functionality. The same could be said about a deleted method. I believe these cases should have their own individual consideration.

Ambiguous Conclusions. The authors conclude that, “the change type Delete Method draws more discussed and relevant questions from developers in the community, and the change type Add Method receives more questions and views from developers.” These results provide insight into what developers are discussing; however, they do not make clear why developers are discussing these change types. Knowing why is critical if you expect API providers to find this research useful for conducting best practices. Yet, the authors state simply that “understanding the developers’ discussion regarding change types is useful for API providers to conduct better practices.”

Case studies may be a good approach to begin exploring why developers are making these posts. Even just 3–5 case studies would API providers some jumping off points for exploring curtailments to the negative effects of API evolution.

Conclusion

In *How Do Developers React to RESTful API Evolution?*, the authors collected data from 11 publicly available APIs to analyze how evolution of these APIs affected developer reactions on Stack Overflow. They found that *adding new methods* to APIs results in the most questions and views per change, whereas *deleting methods* results in the most engagement. This information is useful for public API providers to minimize the impact of future updates on client developers.

To further this area of research, future authors should consider data from a larger variety of sources. They should also dig deeper into developer reactions through case studies. Case studies would provide insights into why client developer engagement differs across change types and what lessons API providers should learn as a result.